Project Development Phase
Model Performance Test

Date	01.04.2026
Team ID	69bf8031c1eab70902258949
Project Name	Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets
Maximum Marks	

1. Testing UI Policies for Mandatory Fields

When you create a UI Policy to enforce mandatory fields, your testing should verify not just that the red asterisk appears, but that the system actually prevents submission.

Core Test Cases

Test Scenario	Expected Result
Condition Met	Field becomes mandatory (red asterisk appears).
Condition Not Met	Field is not mandatory (asterisk disappears).
Reverse if False	If the condition changes from True to False, the field should stop being mandatory.
Form Submission	Attempting to save/submit with a blank mandatory field triggers a browser-level error.
Field Hiding	If a mandatory field is hidden by the policy, it **must** be marked non-mandatory first, or the form will fail to submit.

The "Defense in Depth" Strategy

UI Policies are **client-side only**. They do not stop data from being imported via Excel, API, or List View.

- **Pro Tip:** If the data is critical, create a **Data Policy** as well. You can check the "Use as UI Policy" box on the Data Policy to enforce the rule at both the UI and Server levels simultaneously.

2. Migrating Changes with Update Sets

Update Sets capture configuration changes (UI Policies, Actions, Scripts) but do not capture "data" (actual records in a table).

Workflow for Migration

- **In DEV:** Create and name your Update Set (e.g., `US-INC-001-MandatoryFields`). Ensure it is your **Current** update set.
- **Configuration:** Build your UI Policy and the associated "UI Policy Actions" (where the mandatory setting lives).
- **Completion:** Change the state to **Complete**.
- **In TEST/PROD:** * **Retrieve:** Pull the update set from the source instance.
    - **Preview:** This is the most critical step. Look for **collisions** or **errors**.
    - **Commit:** Apply the changes to the target instance.

Common Migration Pitfalls

- **Missing Actions:** Sometimes developers move the UI Policy but forget to capture the "UI Policy Action" records. Always check the **Customer Updates** related list in your Update Set to ensure both the Policy and its Actions are listed.
- **Order of Operations:** If you have multiple UI Policies affecting the same field, the one with the **highest Order number** wins. Test the "Order" logic in DEV before migrating.
- **Default Update Set:** If you forget to set your new Update Set as "Current," your changes will land in the "Default" set and won't be easily movable. You can manually move them, but it’s messy.

3. Post-Migration Verification

Once committed in the new environment:

- **Clear Cache:** Type `cache.do` in the filter navigator if changes aren't appearing immediately (use with caution in PROD).
- **Functional Testing:** Re-run your "Core Test Cases" above to ensure no other existing Client Scripts or Policies in the new environment are overriding your changes.

REFERENCE LINK:

- **https://youtu.be/jvJaFGXg9X8**